BUILD BRIEF · POWER AUTOMATE

The flow that turns Alert Bot's alerts into Teams posts and emails

Alert Bot watches our Data Factory pipelines and the data they load. When something breaks, gets worse, or is fixed, and once each morning, it sends one HTTP POST with a JSON body. Build **one** Power Automate flow that receives every POST, posts the ready-made Teams card to the right channel for that project, and emails the project's owners when the bot asks for it.

Trigger **When an HTTP request is received** (premium) Volume **about 2–30 calls a day**

Contract **schema_version 1.0** Card **Adaptive Card 1.5, ready to post**

Scope The flow doesn't format anything. The bot sends a finished Teams card in the `card` field and plain-text fields for the email. The flow's job is routing, posting, emailing and not posting the same alert twice.

What the bot sends

Every call is `POST`, `Content-Type: application/json`, with a header `x-alertbot-idempotency-key` that is unique per alert (the same key is reused if the bot retries). There are five kinds of call, told apart by the `event` field.

EVENT	WHEN	TEAMS	EMAIL
`incident.opened`	A new problem: a run failed, a table got too few rows, data checks failed, a run is about to miss its deadline.	Post card	If email_owners
`incident.worsened`	An open problem got more serious (higher severity, or more checks failing).	Post card	If email_owners
`incident.resolved`	The problem is fixed (for example the pipeline was re-run successfully).	Post card	Never
`report.morning`	Once a day per project at 08:00 (bot's time zone): a summary of runs, table loads and open problems. Has an extra `report` object.	Post card	Never
`test`	Someone ran the bot's connection test. The card says "Test alert".	Post card	Never

`email_owners` is `true` only for critical incidents that open or get worse (each project can change this). When it is true, `owner_emails` lists the project owners' addresses.

Fields the flow uses

FIELD	TYPE	USE IT FOR
`project`	string	Routing. The project's key, for example `csdw-adf`. Switch on this.
`card`	object	The Teams card, posted as is.
`email_owners`	boolean	Whether to send the email.
`owner_emails`	string[]	Email recipients (can be empty).
`severity`	info · warning · critical	Email subject, priority.
`title`, `project_name`	string	Email subject.
`summary`, `impact`, `likely_cause`, `action`	string	Email body: what happened, what it affects, why, what to do.
`portal_url`	string	"Open in Alert Bot" link in the email.
`incident_id`	integer	Which incident. `0` for the morning report and tests.
`event`, `occurred_at`, `status`	string	Logging, and phase 2 threading.

Other fields (`checks`, `scope`, `confidence`, `teams_channel`, `report` ...) can be ignored. The bot may add fields later; the flow must accept fields it doesn't know.

The flow, step by step

One flow for all projects. Blocks marked RECOMMENDED can follow after go-live, but they prevent the two real risks: duplicate posts and silent failures.

When an HTTP request is received — Trigger

Who can trigger	Anyone (the bot authenticates with the secret URL)
Method	POST
Request Body JSON Schema	Paste the schema from the last section. Leave schema validation off, so a new optional field can never break the flow.
Settings › Concurrency	On, degree of parallelism **1**. Volume is tiny, and one-at-a-time makes the duplicate check reliable.

{} Scope: Try

Control · RECOMMENDED

Contains	Every step below, so the Catch scope can report any failure.

SP Skip duplicates

SharePoint list or Dataverse table · RECOMMENDED

Store	A list "Alert Bot – delivered" with Title = the idempotency key, plus Event, Project, Received.
Get items	Filter `Title eq '<key>'`, top 1. If one is found: **Terminate (Succeeded)**.
Create item	Otherwise record the key, then continue. Optionally delete entries older than 30 days in a weekly flow.

⇆ Switch on project

Control

On	`triggerBody()?['project']`
Cases	One case per project key (see the table below). Each case posts to that project's Teams channel.
Default	**Required.** Posts to a general "Alert Bot – other projects" channel. The bot adds projects on its own when people sign in with access to new Data Factories, so unknown keys will arrive.

T Post card in a chat or channel

Microsoft Teams · in every case

Post as	Flow bot
Post in	Channel
Team, Channel	That project's team and channel
Adaptive Card	`triggerBody()?['card']` (if the field insists on text, use `string(triggerBody()?['card'])`)

? Condition: email the owners?

Control · after the Switch

All of	`triggerBody()?['email_owners']` is equal to `true` `length(triggerBody()?['owner_emails'])` is greater than `0`

IF YES

Send an email (V2) — Office 365 Outlook

From	A shared mailbox, for example alertbot@... (Send as permission for the flow's account)
To	`join(triggerBody()?['owner_emails'], ';')`
Subject	See expressions
Body	HTML: summary, impact, likely cause, suggested action, and an "Open in Alert Bot" link to `portal_url`
Importance	High

IF NO

Nothing. Resolved incidents, morning reports and tests go to Teams only.

Scope: Catch — Configure run after: has failed, has timed out — RECOMMENDED

Does	Emails the flow's maintainers with the event, project, idempotency key and a link to the failed run. The bot has already had its answer by then, so without this a failed Teams post goes unnoticed.

Project routing

PROJECT (SWITCH CASE)	SHOWN AS	TEAMS CHANNEL
`csdw`	CSDW (Snowflake feeds)	CSDW Data Alerts
`csdw-adf`	CSDW Data Factory	CSDW Data Alerts
`adf-adfpipelinerohan`	adfpipelinerohan (Data Factory)	To be decided by the team
anything else	Projects the bot finds later	Default channel

Note The Teams action needs the team and channel picked at design time; the bot's `teams_channel` field is only a label. Routing is done by the Switch on `project`.

Answering the bot

- Don't add a **Response** action. The trigger then answers `202 Accepted` immediately, which the bot treats as delivered.

- If you add one anyway, answer `200` straight away, before the Teams and email steps.
- Never answer `4xx` for something the flow doesn't recognise (an unknown project, a new event). The bot treats 4xx as "rejected, don't retry" and the alert is lost. Route unknowns to the default channel instead.

Expressions

Switch on	`triggerBody()?['project']`
Teams card	`triggerBody()?['card']`
Idempotency key	`triggerOutputs()?['headers']?['x-alertbot-idempotency-key']`
Email to	`join(triggerBody()?['owner_emails'], ';')`
Email subject	`concat('[', toUpper(triggerBody()?['severity']), '] ', triggerBody()?['project_name'], ': ', triggerBody()?['title'])`
Email condition	`and(equals(triggerBody()?['email_owners'], true), greater(length(coalesce(triggerBody()?['owner_emails'], json('[]'))), 0))`

Email body suggestion: a heading with the title, then four short paragraphs labelled *What happened* (`summary`), *Impact* (`impact`), *Likely cause* (`likely_cause`) and *What to do* (`action`), then a button-style link to `portal_url` . These fields are plain text; the flow should not need to clean them.

How the bot behaves

TOPIC	BEHAVIOUR
When it sends	Only when an incident opens, gets worse or resolves, plus one morning report per project per day. Nothing is sent while an owner has muted the project or a check (the incident is still recorded).
Success	Any 2xx answer.
Timeout	15 seconds per attempt.
Retries	On a timeout, network error, `429` or `5xx`: 3 quick attempts (2 s and 4 s apart), then again on the bot's next rounds (every 10–15 minutes), up to 6 rounds. After that the alert is marked failed and shown on the bot's Alerts page.
Not retried	`4xx` other than 429: treated as a permanent rejection.
Duplicates	Possible: if the flow accepted a call but the answer was lost, the bot sends it again with the same idempotency key. Hence the "skip duplicates" step.
Order	Events for one incident are sent in the order they happened, but a retried call can arrive after a newer one. Don't assume order.
Size	An incident is typically 3–8 KB; the morning report is larger because it carries the full report.

Ownership, licensing and security

- **Licence.** The HTTP trigger is a premium connector: the flow's owner needs Power Automate Premium (per user), or the flow needs a Process licence. Without premium, see the fallback below.
- **Owner.** Build and own the flow with a service account, not a person, and make the Teams and Outlook connections with that account. If the flow belongs to someone who leaves, alerts stop.
- **Co-owners.** Add at least one more person as co-owner so it can be maintained.
- **Teams.** The service account must be a member of each target team, and the Workflows / Power Automate app must be allowed in those teams so "Flow bot" can post.
- **The URL is a password.** The trigger URL contains a signature (`sig=…`) that lets anyone who has it post into our channels. Send it to the bot's administrator privately, never in a ticket, chat or document. If it leaks, regenerate it (Settings › regenerate the trigger, or save the flow as a copy) and send the new one.
- **No secrets in the flow.** The payload holds no passwords or data rows. It does hold pipeline names, table names and owners' email addresses, so keep the flow's run history at the default retention and restrict who can view the flow.

Before go-live A trigger URL for the current test flow has already been shared in a chat. Regenerate it, or build the new flow and give the bot the new URL, before real alerts go out.

Fallback without premium

In the Teams channel, add the **Workflows** template "Post to a channel when a webhook request is received" and give its URL to the bot's administrator. The bot is switched to `notifier: teams_workflow` and sends only the card, which Teams posts. This needs no flow building, but it means one URL per channel and no emails.

Phase 2 (optional)

- **Thread follow-ups.** Save the Teams message id returned by the post with `project` + `incident_id` (in the same SharePoint list). For `incident.worsened` and `incident.resolved`, use "Reply with an adaptive card in a channel" under the original post, so each incident is one conversation.
- **Mention on-call.** For critical `incident.opened`, @mention the on-call person or a tag.
- **Quiet hours.** Hold non-critical events outside working hours and post them as a digest.

Testing and sign-off

1. Before the bot is connected, POST the sample payload to the trigger URL with Postman or curl, changing `project` and `event` to try each case:
   `curl -X POST "<trigger URL>" -H "Content-Type: application/json" -H "x-alertbot-idempotency-key: try-1" --data @sample.json`
2. Then the bot's administrator puts the URL in the bot and runs its connection test for each project (`alertbot notify-test csdw-adf`, and the same for the other keys). Each posts a card titled "Test alert from Alert Bot".

0 of 10 checked

- [ ] **Test event** posts the card in the right channel for each project key.
- [ ] **Unknown project** (e.g. `project: "zzz"`) posts in the default channel and the call still returns 2xx.
- [ ] **Critical incident.opened** with `email_owners: true` posts the card and emails everyone in `owner_emails`.
- [ ] **Warning incident.opened** (`email_owners: false`) posts the card and sends no email.
- [ ] **incident.resolved** posts the card and sends no email.
- [ ] **report.morning** posts the card (it has no email, and `incident_id` is 0).
- [ ] **Same idempotency key twice** results in one post only.

- [ ] **A new field** added to the JSON doesn't break the flow.
- [ ] **A failure** (e.g. a wrong channel) reaches the maintainers through the Catch scope.
- [ ] **The trigger answers in under 2 seconds** (202 without a Response action).

Handover

When the checklist passes, give the bot's administrator:

1. The trigger URL, privately. It goes into the bot's settings file as `PA_CSDW_URL` (all projects use it today), never into the bot's project files.
2. The project → channel table as built, including the default channel.
3. The service account and co-owners of the flow, and where failures are emailed.

Sample payload and schema

A real `incident.opened` call, generated by the bot's own code with example values. The `card` is exactly what will be posted.

sample.json · incident.opened (example values)


```
{
  "schema_version": "1.0",
  "event": "incident.opened",
  "occurred_at": "2026-10-14T07:42:10Z",
  "project": "csdw-adf",
  "project_name": "CSDW Data Factory",
  "incident_id": 42,
  "severity": "critical",
  "status": "open",
  "title": "DEV-TRADE_STAT failed",
  "summary": "DEV-TRADE_STAT failed at step 'Copy trade stats' after 4 min. 0 of the usual ~12,400
rows reached TRADE_STATISTICS.",
  "impact": "TRADE_STATISTICS has no rows for today; 2 pipelines that read it will run on yesterday's
data.",
  "likely_cause": "The source file for today wasn't in the inbound folder when the copy ran.",
  "action": "Check that today's file arrived in tradestats/inbound, then re-run the pipeline.",
  "confidence": "high",
  "analysis_source": "ai",
  "checks": [
    "run_failed",
    "row_count"
  ],
  "scope": "site:DEV-TRADE_STAT",
  "opened_at": "2026-10-14T07:42:10Z",
  "resolved_at": null,
  "portal_url": "https://alertbot.example.com/p/csdw-adf/i/42",
  "teams_channel": "CSDW Data Alerts",
  "email_owners": true,
  "owner_emails": [
    "owner.one@example.com",
```

```
      "owner.two@example.com"
    ],
    "card": {
      "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
      "type": "AdaptiveCard",
      "version": "1.5",
      "msteams": {
        "width": "Full"
      },
      "body": [
        {
          "type": "Container",
          "style": "attention",
          "bleed": true,
          "items": [
            {
              "type": "TextBlock",
              "text": "🔴 New incident · CSDW Data Factory",
              "size": "Small",
              "weight": "Bolder",
              "isSubtle": true,
              "wrap": true
            },
            {
              "type": "TextBlock",
              "text": "DEV-TRADE_STAT failed",
              "size": "Large",
              "weight": "Bolder",
              "wrap": true
            }
          ]
        },
        {
          "type": "FactSet",
          "facts": [
            {
              "title": "Severity",
              "value": "Critical"
            },
            {
              "title": "Scope",
              "value": "DEV-TRADE_STAT"
            },
            {
              "title": "Checks",
              "value": "run failed, row count"
            },
            {
              "title": "Incident",
              "value": "#42"
            }
          ]
        },
        {
          "type": "TextBlock",
          "text": "What happened",
          "weight": "Bolder",
          "spacing": "Medium"
        },
        {
          "type": "TextBlock",
```

```
        "text": "DEV-TRADE_STAT failed at step 'Copy trade stats' after 4 min. 0 of the usual ~12,400
rows reached TRADE_STATISTICS.",
        "wrap": true
      },
      {
        "type": "TextBlock",
        "text": "Likely cause",
        "weight": "Bolder",
        "spacing": "Medium"
      },
      {
        "type": "TextBlock",
        "text": "The source file for today wasn't in the inbound folder when the copy ran.",
        "wrap": true
      },
      {
        "type": "TextBlock",
        "text": "Suggested action",
        "weight": "Bolder",
        "spacing": "Medium"
      },
      {
        "type": "TextBlock",
        "text": "Check that today's file arrived in tradestats/inbound, then re-run the pipeline.",
        "wrap": true
      },
      {
        "type": "TextBlock",
        "isSubtle": true,
        "size": "Small",
        "wrap": true,
        "spacing": "Medium",
        "text": "AI analysis · confidence high"
      }
    ],
    "actions": [
      {
        "type": "Action.OpenUrl",
        "title": "Open in portal",
        "url": "https://alertbot.example.com/p/csdw-adf/i/42"
      }
    ]
  }
}
```

The trigger's **Request Body JSON Schema**. It names every field so they appear as dynamic content; nothing is marked required.

Request Body JSON Schema

```
{
  "type": "object",
  "properties": {
    "schema_version": {
      "type": "string",
      "description": "Payload contract version, currently 1.0"
    },
    "event": {
      "type": "string",
      "enum": [
```

```
        "incident.opened",
        "incident.worsened",
        "incident.resolved",
        "report.morning",
        "test"
      ]
    },
    "occurred_at": {
      "type": "string",
      "description": "ISO-8601 UTC"
    },
    "project": {
      "type": "string",
      "description": "Project key, e.g. slipstream; a shared flow switches on this"
    },
    "project_name": {
      "type": "string"
    },
    "incident_id": {
      "type": "integer",
      "description": "0 for report.morning and test (they are not about one incident)"
    },
    "severity": {
      "type": "string",
      "enum": [
        "info",
        "warning",
        "critical"
      ]
    },
    "status": {
      "type": "string",
      "enum": [
        "open",
        "acknowledged",
        "resolved"
      ]
    },
    "title": {
      "type": "string"
    },
    "summary": {
      "type": "string",
      "description": "What happened, with the numbers"
    },
    "impact": {
      "type": "string"
    },
    "likely_cause": {
      "type": "string"
    },
    "action": {
      "type": "string",
      "description": "Suggested next step"
    },
    "confidence": {
      "type": "string",
      "enum": [
        "low",
        "medium",
        "high",
```

```
        ""
      ]
    },
    "analysis_source": {
      "type": "string",
      "enum": [
        "rules",
        "ai"
      ]
    },
    "checks": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "scope": {
      "type": "string",
      "description": "site:<name> or output:<id>"
    },
    "opened_at": {
      "type": [
        "string",
        "null"
      ]
    },
    "resolved_at": {
      "type": [
        "string",
        "null"
      ]
    },
    "portal_url": {
      "type": "string",
      "description": "Deep link to the incident in the portal"
    },
    "teams_channel": {
      "type": "string"
    },
    "email_owners": {
      "type": "boolean",
      "description": "True when this event should also be emailed (by default: critical
opened/worsened)"
    },
    "owner_emails": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "report": {
      "type": "object",
      "description": "report.morning only: the full report data (status, per-pipeline and per-table
figures)"
    },
    "card": {
      "type": "object",
      "description": "Ready-to-post Adaptive Card 1.5 for Teams"
    }
```

```
    }
  }
```